ZENIKA · SPEED TRAINING IA · 4H

Développeur IA — Avancé

Sous le capot des agents de code

Le modèle prédit, le harness agit, la gateway gouverne.

Agenda

0 Ouverture express — 15 min

1 Sous le capot — 25 min

2 Le harness — 45 min

3 Ingénierie du contexte — 25 min, *puis pause 10 min*

4 Spec-driven et qualité — 45 min

5 Orchestrer et paralléliser — 35 min

6 Au-delà du poste — 25 min, *puis clôture 10 min*

© ZENIKA 2026 | All rights reserved

Module 0 · 15 min

Ouverture express

Où en est-on, et quel vocabulaire partage-t-on ?



© ZENIKA 2026 | All rights reserved

Quiz de positionnement

QUIZ 3 min

1

Un cache expiré fait-il oublier quelque chose à l'agent ?

Non : on repaie, rien n'est oublié

2

Skill ou hook : lequel s'exécute à coup sûr ?

Le hook

3

Spec-first ou spec-anchored : laquelle reste à jour ?

Spec-anchored

4

10 agents fiables à 95 %, enchaînés : quelle fiabilité ?

Environ 60 %

© ZENIKA 2026 | All rights reserved

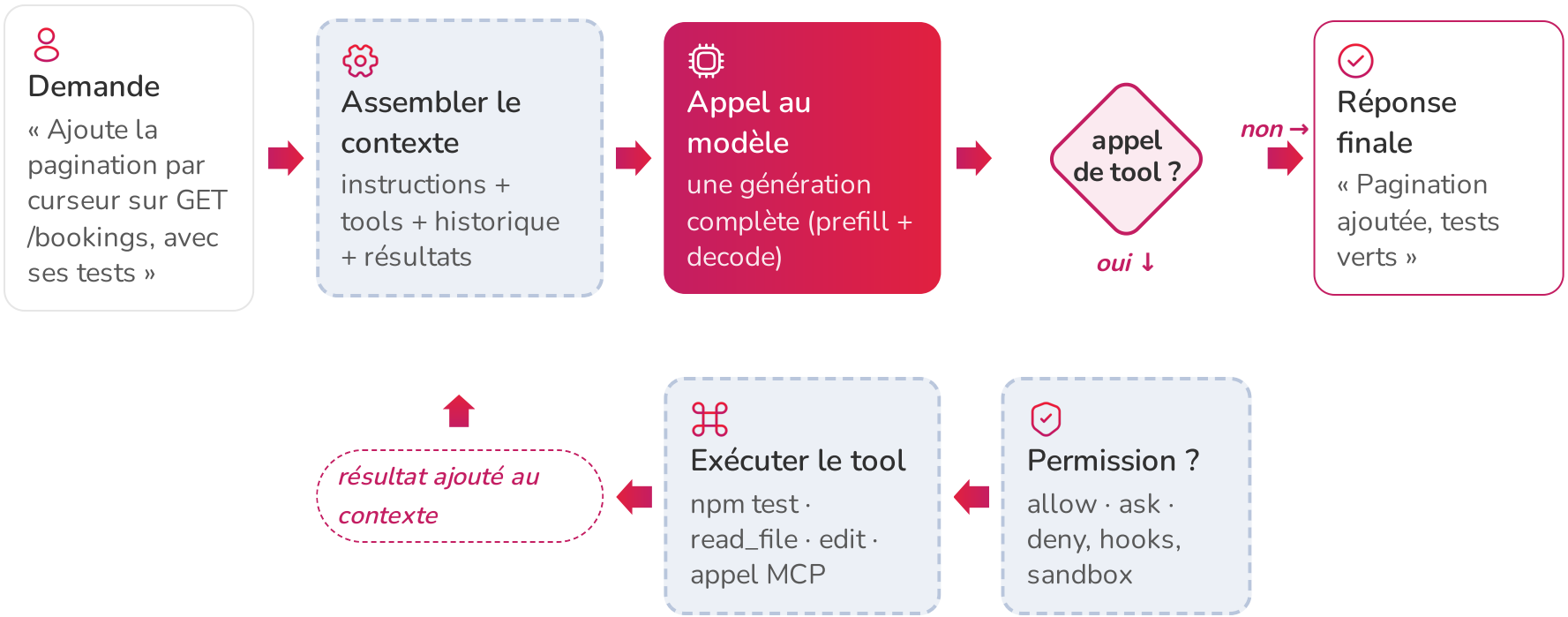
Vibe coding vs AI-driven dev

	Vibe coding	AI-driven dev
Logique	explorer vite	produire de façon fiable
Pour	spike, POC jetable, script ponctuel	fonctionnalité livrée dans un code existant
Exemple	« fais-moi une API de démo »	paginer GET /bookings, avec ses tests
Réflexe	on tente, on observe, on jette	on cadre, on vérifie, on capitalise

Viber pour explorer, piloter pour livrer. Cette formation est du côté droit.

© ZENIKA 2026 | All rights reserved

La boucle, tour par tour



Un tour = un appel modèle complet. Une tâche simple : 5 à 20 tours. Une grosse tâche : des centaines.

© ZENIKA 2026 | All rights reserved

Module 1 · 25 min

Sous le capot

Pourquoi un agent coûte cher, est lent ou se trompe ?



© ZENIKA 2026 | All rights reserved

Qui parle à qui : la pile complète

Méthode : SDD spec.md → plan.md → tasks.md, relus par l'humain

Orchestrateur découpe, distribue, attend, agrège, reprend sur erreur

Agent A	Agent B	Agent C
harness : boucle, tools, contexte	harness : boucle, tools, contexte	harness : boucle, tools, contexte
appel au modèle	appel au modèle	appel au modèle

Gateway + guardrails auth, routage, quotas, cache, journalisation, filtrage entrée / sortie

LLM cloud	Hyperscaler	Souverain	SLM local
Anthropic, OpenAI...	*Bedrock, Vertex, Azure*	*Mistral, Scaleway...*	*Ollama, vLLM*

L'humain écrit et valide la spec, puis supervise aux points clés.

Outils

fichiers, terminal, git, navigateur

serveurs MCP : GitHub, Jira, BDD...

exécutés par le harness, jamais par le modèle

*Point de passage obligé : c'est là que l'entreprise **contrôle**.*

© ZENIKA 2026 | All rights reserved

Harness vs orchestrateur : la frontière

HARNESS

Fait d'un modèle un agent

- Gère une boucle d'agent
- Exécute les tools, applique les permissions
- Construit et compacte le contexte

Claude Code | Codex CLI | Copilot CLI | OpenCode

ORCHESTRATEUR

Fait travailler plusieurs agents

- Décompose et route les tâches
- Parallélise, attend, agrège
- Persiste l'état, reprend après panne

LangGraph | CrewAI | OpenAI Agents SDK | Temporal

La frontière bouge : retenez le **rôle**, pas le produit.

© ZENIKA 2026 | All rights reserved

KV cache : ne pas tout recalculer à chaque token

1 Prefill : lire le prompt

Tous les tokens du prompt traités en parallèle.

`Le` `chat` `dort` `sur` `le`

KV cache : K et V de chaque token

- limité par le calcul (GPU saturé)
- fixe le temps avant le premier token (TTFT)

2 Decode : écrire la réponse

Un seul nouveau token par passe.

`Le` `chat` `dort` `sur` `le` **`canapé`**

cache réutilisé tel quel + K, V de « canapé »

- limité par la bande passante mémoire
- fixe les tokens par seconde

SANS CACHE

Chaque token recalculerait K et V de tous les précédents : un coût quadratique.

LE PRIX

Llama 3 8B : 128 Ko par token, 16 Go pour 128 k tokens.

LES PARADES

Têtes K/V partagées, cache quantifié, mémoire paginée.

© ZENIKA 2026 | All rights reserved

Du KV cache au prompt caching

LE PRINCIPE

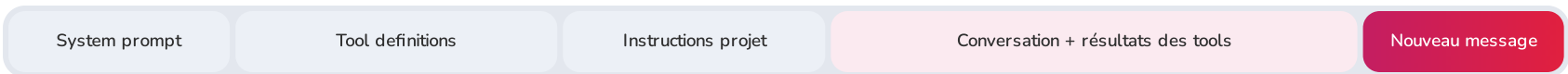
Le fournisseur garde le KV cache d'un préfixe déjà vu

Même début de requête : le prefill est sauté. Plus rapide, **beaucoup moins cher**.

~90 %

de remise sur l'entrée lue en cache

ANATOMIE D'UNE REQUÊTE D'AGENT : DU PLUS STABLE AU PLUS VOLATIL



← *stable : servi par le cache d'un tour à l'autre*

volatil : seule la fin est recalculée →

CACHE EXPIRÉ

L'agent n'a **rien oublié** : la conversation est renvoyée en entier.

MAIS

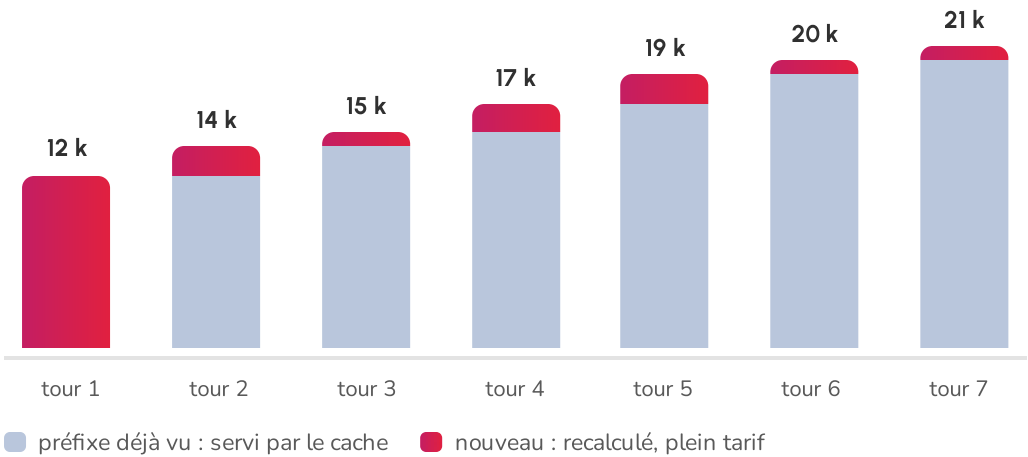
Le préfixe est **repayé plein tarif**, et la réponse arrive plus tard.

DURÉE DE VIE

De quelques minutes à quelques heures sans activité.

© ZENIKA 2026 | All rights reserved

Ce que coûte cette trace



118 k tokens d'entrée envoyés au total, dont 82 % lus en cache

LE CONSTAT

L'entrée fait le volume

Chaque tour renvoie tout le préfixe déjà vu, et n'ajoute que quelques milliers de tokens.

AVEC LE PROMPT CACHING

Ce préfixe est lu en cache, à ~10 % du prix.

LA SORTIE

~5× plus chère au token, mais petite en volume.

i Réduire ce qui **entre** rapporte plus que raccourcir la **sortie**.

© ZENIKA 2026 | All rights reserved

Tokens de raisonnement : choisir l'effort

LE PRINCIPE

Le modèle « réfléchit » en tokens avant de répondre

Ces **reasoning tokens** sont facturés comme de la **sortie**, et souvent invisibles.

×5

prix d'un token de sortie face à l'entrée



EFFORT BAS · MODÈLE LÉGER

Renommer, documenter, appliquer un pattern.



EFFORT MOYEN · GÉNÉRALISTE

Paginer un endpoint et ses tests, relire un diff.



EFFORT HAUT · RAISONNEMENT

Bug difficile, migration, architecture.

i L'effort se choisit **par tâche** : plus lent, plus cher, et **pas toujours meilleur**.

© ZENIKA 2026 | All rights reserved

Les SLM : petits, locaux, spécialisés

DÉFINITION PRATIQUE

De ~1 à ~15 milliards de paramètres

Assez petit pour tourner sur un poste ou un seul GPU.



EXEMPLES

Gemma 4 | Qwen3.5 | Phi-4-mini | Granite 4 | Ministral



BONS CAS D'USAGE

Classer, extraire, router, compléter du code, données qui ne sortent pas.



LIMITES

Raisonnement long, agents autonomes sur de gros dépôts.



PATTERN HYBRIDE

Le SLM trie et route, le LLM raisonne.

© ZENIKA 2026 | All rights reserved

Quiz : où part la facture ?

QUIZ 3 min

System prompt et tool definitions

stable : en cache

Log de build de 3 000 lignes

payé, puis renvoyé à chaque tour

Nouvelle demande

plein tarif, mais petite

Tokens de raisonnement

prix de sortie, invisibles

TIP Garde le début du contexte stable : c'est lui que le cache rentabilise.

© ZENIKA 2026 | All rights reserved

Module 2 · 45 min

Le harness

Quelles briques entourent le modèle, et laquelle utiliser pour quoi ?



© ZENIKA 2026 | All rights reserved

Les capacités attendues d’un harness



Sous-agents

Contexte isolé, et le droit d’écrire.



MCP standard

N’importe quel serveur, déclaré dans un fichier versionné.



Instructions, skills, hooks

Personnaliser sans toucher au cœur.



Modes de pilotage

Monter l’autonomie, borner les droits.



Scriptable

Un mode non interactif, pour la CI.

Plateforme intégrée ou **composition** d’extensions : le harness est un **actif d’équipe versionné**. Sinon, c’est de l’**agentic debt**.

© ZENIKA 2026 | All rights reserved

Une allowlist n’est pas une sandbox

Allowlist

CE QU’ELLE FILTRE
Des motifs de commande

CONTOURNEMENT
npm test && curl …, un script qui en appelle un autre

FORME
Une liste dans la config du harness

Sandbox

CE QU’ELLE BORNE
Les effets : fichiers, réseau, droits

CONTOURNEMENT
Aucun par le prompt : l’agent ne sort pas du périmètre

FORME
Conteneur, VM, compte dédié, réseau coupé

Le seul périmètre réel est l’**isolation**.

© ZENIKA 2026 | All rights reserved

Les primitives du harness : qui décide ?



Instructions projet

AGENTS.md : règles stables, conventions, commandes.

QUI DÉCIDE

personne : toujours chargées



Skill

Savoir-faire réutilisable, chargé quand la tâche s'y prête.

QUI DÉCIDE

le modèle, sur sa description



Hook

Script lancé à un moment précis du cycle de vie.

QUI DÉCIDE

le harness, à chaque événement



Sous-agent

Travail à part, ne renvoie que son résultat.

QUI DÉCIDE

l'agent ou toi



MCP

Accès à un système externe.

QUI DÉCIDE

le modèle, à la demande

i Le critère qui range tout : **qui décide** qu'une primitive s'applique.

© ZENIKA 2026 | All rights reserved

Hooks : automatiser le cycle de vie

Des **scripts déterministes**, lancés par le harness, quoi que dise le modèle.

AVANT UN TOOL

Bloquer

git push --force, rm -rf, ou réécrire une commande verbeuse.

APRÈS UN TOOL

Formater

Le fichier écrit, ou journaliser l'appel.

FIN DE TOUR

Vérifier

Lancer les tests, relire le diff produit.

FIN DE SESSION

Archiver

La trace, ou proposer une mise à jour des notes.

i « Lance toujours les tests » dans les instructions est une **suggestion**. Le hook est une **garantie**.

© ZENIKA 2026 | All rights reserved

Tool calling : ce qui transite réellement

① Le harness déclare

```
{
  "name": "run_tests",
  "description":
    "Lance les tests",
  "input_schema": {
    "type": "object",
    "properties": {
      "path": {
        "type": "string"
      }
    }
  }
}
```

② Le modèle demande

```
{
  "type": "tool_use",
  "id": "call_01",
  "name": "run_tests",
  "input": {
    "path": "tests/bookings.test.ts
  }
}
```

③ Le harness répond

```
{
  "type": "tool_result",
  "tool_use_id": "call_01",
  "content":
    "FAIL bookings:42
     nextCursor is
     undefined"
}
```

i Le modèle choisit **sur la description** : chaque tool definition part **en tokens** à chaque appel.

© ZENIKA 2026 | All rights reserved

Ce que le harness injecte



SESSION AGENTIQUE · LE HARNESS INJECTE BIEN PLUS QUE VOS MESSAGES

Sorties de commandes

Un build, un npm install ou une stack trace peuvent dominer la context window s'ils ne sont pas filtrés.

La question n'est pas « l'agent est-il intelligent ? », mais « **ce qu'il a injecté aide-t-il la tâche ?** »

© ZENIKA 2026 | All rights reserved

Quiz : quelle primitive ?

QUIZ 3 min

« Toujours passer par le client HTTP interne »

instructions

Interdire git push --force, quoi qu'il arrive

hook

Migrer le schéma, une fois par mois

skill

Explorer 40 fichiers sans polluer la session

sous-agent

© ZENIKA 2026 | All rights reserved

Exercice 1 — Garde-fou déterministe

OPTIONNEL 20 min ↗

Le concept

Ce qui doit **toujours** arriver est un **hook**, pas une consigne.

La consigne

1. Choisis un besoin du quiz : bloquer une commande, lancer les tests…
2. Mets-le en place **avec le mécanisme de hook de ton harness**.
3. Demande ensuite à ton agent de faire **le contraire**.

Réussi quand…

- l'action est **bloquée**, ou la vérification **lancée**, par le harness ;
- **même** quand on demande le contraire.

Débrief : qu'a tenté l'agent pour passer outre ?

TIP Ce qui doit toujours arriver est un hook, pas une consigne.

© ZENIKA 2026 | All rights reserved

Module 3 · 25 min

Ingénierie du contexte

Comment donner à l'agent le bon contexte, au bon moment, au bon coût ?



© ZENIKA 2026 | All rights reserved

Ce qui entre dans la context window

Élément	Entre quand ?	Ensuite
AGENTS.md, tool definitions	au démarrage	dans chaque appel, en cache
Historique	à chaque échange	grossit, compacté près de la limite
Fichiers lus, sorties de tools	à la demande	renvoyés jusqu'à la fin de la session
Skills	quand le modèle les juge utiles	nom et description d'abord, contenu ensuite

i Pour le savoir : **regarder** la jauge, **demander** à l'agent ce qu'il a lu, **élaguer** ce qui ne sert plus.

© ZENIKA 2026 | All rights reserved

KIT DE CODE

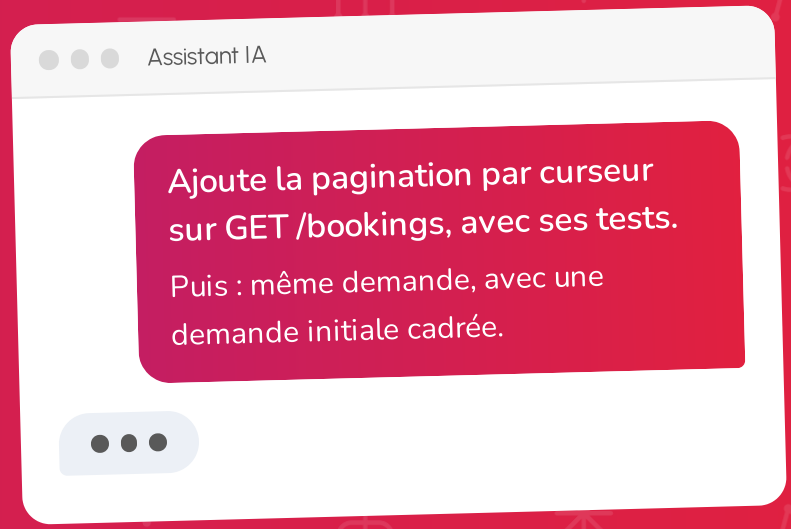
▶ DÉMO

Avec ou sans demande initiale

La même tâche, deux sessions : on compte les fichiers lus.

Demande initiale : objectif, fichiers clés, commandes, invariants, pièges.

Le gain annoncé n'est pas le gain mesuré : on compte.



© ZENIKA 2026 | All rights reserved

Réduire les tokens : un levier par buffer

SORTIES DE TOOLS

Filtrer

Erreurs seules, dernières lignes.

DONNÉES LOURDES

Traiter hors contexte

Un script, un index.

ENTRE SESSIONS

Mémoire ciblée

Ne plus tout réexpliquer.

RÉPONSE

Demander du concis

Pas de préambule.

CODE GÉNÉRÉ

En écrire moins

Moins à relire ensuite.

i Comprimer la prose du modèle rapporte peu : l'**entrée** domine. Faire **écrire moins de code** rapporte deux fois : maintenant et à la relecture.

© ZENIKA 2026 | All rights reserved

Le piège des sous-agents

Un cas vécu

Une demande de recherche un peu large

↓

6 sous-agents lancés en parallèle

↓

Chacun charge toutes les pages qu'il lit

↓

1 M de tokens en 30 min, quota épuisé

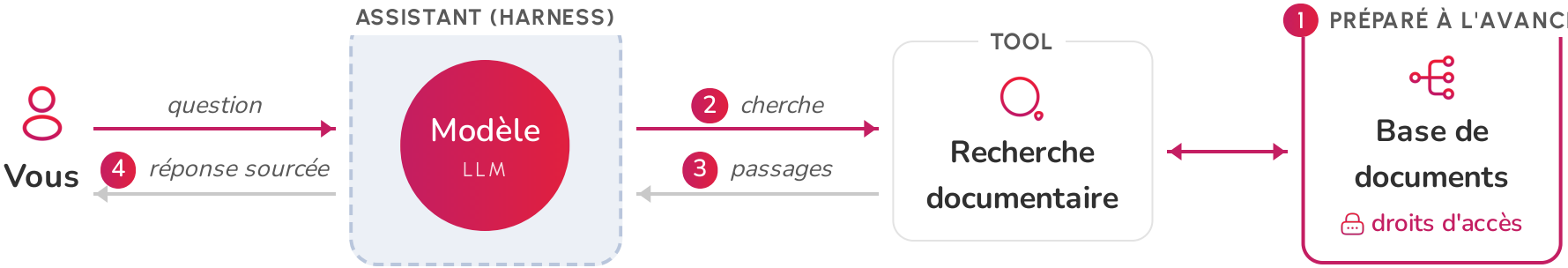
Ce qu'on retient

- La jauge de la session principale allait **très bien**.
- Elle ne montre pas ce que consomment les sous-agents.
- On borne **périmètre** et **nombre d'agents** avant de lancer.

i Un sous-agent protège ta context window, **pas ton budget**.

© ZENIKA 2026 | All rights reserved

RAG : le retour des embeddings



Exemple : « Quelle est notre convention de pagination pour les API ? »

1

Indexer *(indexing)*

La doc est découpée en morceaux ; un modèle d'embedding calcule un vecteur par morceau.

> *Wiki technique et ADR indexés.*

2

Chercher *(retrieval)*

La question est vectorisée ; on prend les morceaux les plus proches.

> *Les 3 passages sur la pagination.*

3

Augmenter *(augmentation)*

Ces morceaux entrent dans le contexte, avec la question.

> *3 passages, pas les 400 pages du wiki.*

4

Générer *(generation)*

Le modèle répond à partir des passages et cite ses sources.

> *« Curseur opaque, 100 max [ADR-012] »*

© ZENIKA 2026 | All rights reserved

La mémoire de l’agent

FICHIER VERSIONNÉ

Instructions et notes du projet

Relues en PR, comme du code.

MÉMOIRE DU HARNESS

L'agent écrit ses notes

Propriétaire, souvent plafonnée : à relire.

MIDDLEWARE TIERS

Serveur MCP ou plugin

Indexé, à votre main, exportable.

LE BON MÉCANISME : PROGRESSIVE DISCLOSURE

Chercher dans un index compact, puis ne charger que le détail des entrées retenues : ~10× moins de tokens.

Une mémoire qu’on ne peut pas **relire** finit par contenir des faits périmés.

© ZENIKA 2026 | All rights reserved

Exploiter les traces

Une session contient

- ce qui a été **tenté** et ce qui a **échoué**
- ce qui **manquait** : doc, tool, convention
- ce qui a **fini par marcher**

C'est de l'**inférence déjà payée**.

Une extraction ciblée

Fichier	Contenu
`troubleshooting.md`	erreur → cause → fix
`api-fixes.md`	noms, docs à corriger
`next-session.md`	ce qu'on recharge

Non relue, une extraction n'est jamais une source d'autorité.

TIP Donne à l'agent de quoi chercher, pas tout le dépôt.

© ZENIKA 2026 | All rights reserved

Pause · 10 min

On souffle



© ZENIKA 2026 | All rights reserved

Module 4 · 45 min

Spec-driven et qualité

Comment garder le code aligné sur l'intention, et le prouver ?



© ZENIKA 2026 | All rights reserved

Ce que l'IA change dans notre façon de travailler

L'agent a compressé le temps de **production**, pas celui de **vérification**.

Hier	Avec un agent
RYTHME *Une feature = des jours*	**RYTHME** *Une feature = des heures*
CE QUI GARANTIT *Revues et rituels, a posteriori*	**CE QUI GARANTIT** *Contrôles systématiques, à chaque étape*

Les revues **ne disparaissent pas** : elles ne suffisent plus **seules**. On leur adjoint des **contrôles automatiques**, au rythme de l'agent.

© ZENIKA 2026 | All rights reserved

Trois postures face à la spec

SDD : la spec versionnée est la source de vérité que l'agent implémente et que les tests vérifient.

1



Spec-first

Écrite d'abord, comme prompt structuré. Non maintenue : elle diverge du code.

2



Spec-anchored

Spec et code évoluent ensemble. Les écarts sont détectés.

3



Spec-as-source

Seule la spec est éditée. Le code devient un artefact généré.

i Cible réaliste : **spec-anchored**. L'agent sait **formaliser** une spec, pas décider **ce qui doit être vrai**.

© ZENIKA 2026 | All rights reserved

Spec-centric ou workflow-centric

	Spec-centric	Workflow-centric
Exemples	Spec Kit, OpenSpec, BMAD	Superpowers
Référence	la spec, versionnée	la discipline, encodée en skills
Ce qui survit	un document relisible	un process reproductible
Point fort	mémoire longue du projet	boucle d'implémentation serrée



Pas un concurrent à départager : les deux se **combinent**.

© ZENIKA 2026 | All rights reserved

Garde-fous : du code au commit

L'agent génère vite : sans garde-fou, l'écart avec l'intention passe **inaperçu**.

1

Tests

Le contrat exécutable.

VÉRIFIE

le comportement

2

Revue agentique

Par un agent distinct de celui qui a écrit le code.

VÉRIFIE

le diff, avant d'envoyer

3

Commit

Le point de synchronisation entre code, spec et IA.

VÉRIFIE

la doc suit, la part d'IA est tracée

i **Règle minimale** : aucun code généré ne part en production sans **vérification déterministe** (tests, analyse statique) et **revue par un agent distinct**.

© ZENIKA 2026 | All rights reserved

Tests générés : deux natures, une mesure

TESTS DE CONSTRUCTION

Servent à l'agent

Testent l'implémentation, unité par unité. Jetables.

TESTS FONCTIONNELS

Servent à l'équipe

Testent les effets observables. **Durables** : la spec exécutable.

SCORE DE MUTATION

On modifie le code exprès (`<` devient `⩽`) : la suite doit échouer. Bien plus dur à tricher que la couverture.

i En fin de feature, là où les deux se recouvrent, **le test fonctionnel gagne**.

© ZENIKA 2026 | All rights reserved

Le drift spec ↔ code : qui le détecte ?

Drift : l'écart qui s'installe entre la spec et le code.

Détecteur	Ce qu'il voit	Ce qu'il rate
Tests fonctionnels	un comportement qui contredit une exigence encodée	une exigence nouvelle, sans test
Revue de spec dans la PR	un diff qui ne raconte pas la même chose que la spec	ce que le relecteur n'a pas lu
Contrôle au commit	du code changé sans spec touchée	une spec touchée… mais fausse
Personne	—	tout : le drift est silencieux

i Quand une exigence change, c'est la **revue de spec** qui le voit d'abord.

© ZENIKA 2026 | All rights reserved

Exercice 2 — La spec change

OPTIONNEL 25 min ↗

Le concept

La spec versionnée est la source de vérité ; le drift se détecte, il ne se devine pas.

La consigne

1. Écris une spec courte pour une fonctionnalité de l'API.
2. Versionne-la, fais-la implémenter avec ses tests.
3. En cours de route, change une exigence.

Réussi quand…

- la spec est versionnée dans le dépôt ;
- l'implémentation passe ses tests ;
- le changement est détecté, et on sait dire par quoi.

Débrief : qu'est-ce qui aurait laissé passer le drift ?

TIP Si la spec n'est pas versionnée avec le code, elle ment déjà.

© ZENIKA 2026 | All rights reserved

Module 5 · 35 min

Orchestrer et paralléliser

Quand découper en plusieurs agents, et à quel prix ?



© ZENIKA 2026 | All rights reserved

Cinq patterns d'orchestration

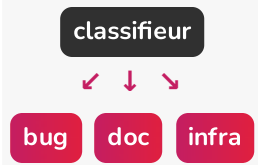
Chaîne



étapes fixes, chaque sortie nourrit la suivante

> *spec → plan → code*

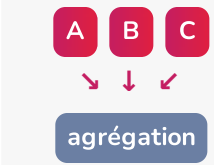
Routeur



on oriente vers le bon agent ou le bon modèle

> *un SLM trie, un LLM traite*

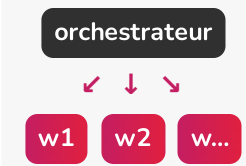
Parallélisation



tâches indépendantes, ou vote entre plusieurs

> *3 revues d'une même PR*

Orchestrateur-workers



le découpage est décidé dynamiquement par un LLM

> *refacto sur 12 modules*

Évaluateur



on itère jusqu'à ce que le critère soit atteint

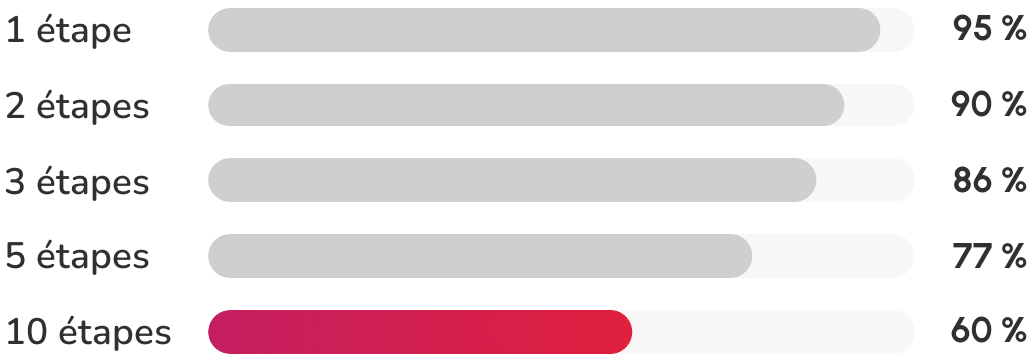
> *code + revue par un 2e agent*

workflow figé : chemin connu à l'avance

workflow dynamique : le LLM pilote davantage

© ZENIKA 2026 | All rights reserved

La fiabilité composée : 10 étapes à 95 % ≈ 60 %



Fiabilité d'une chaîne d'agents fiables à 95 % chacun

LE CALCUL

Les fiabilités se multiplient

0,95 × 0,95 × … dix fois ≈ **0,60**.

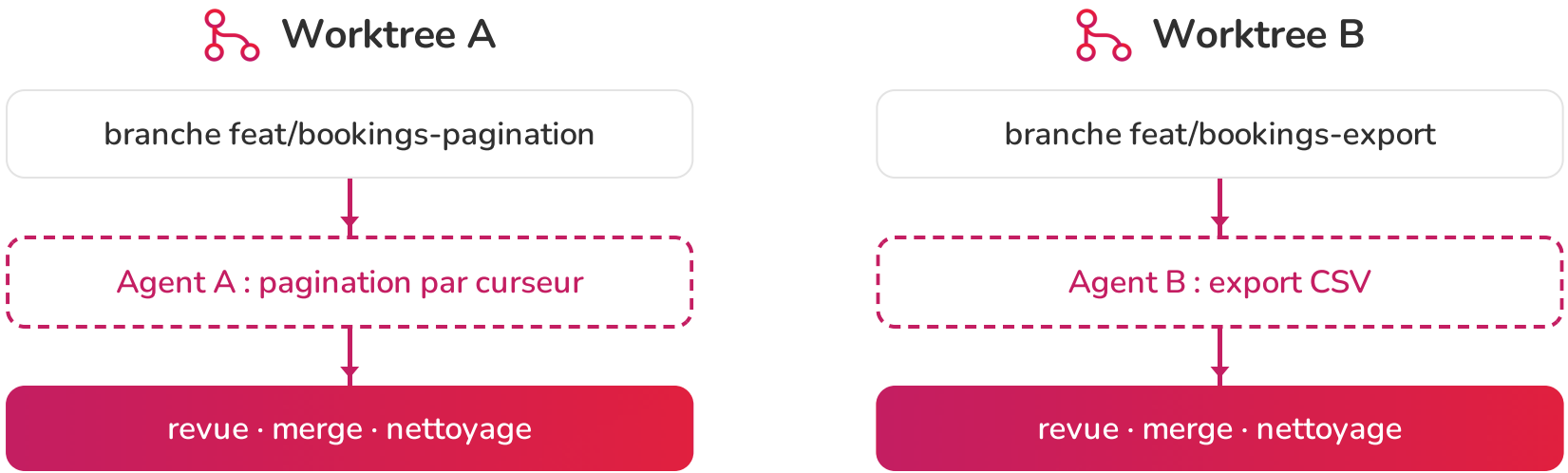
LA PARADE

Une chaîne **courte**, et une **vérification** entre les étapes.

© ZENIKA 2026 | All rights reserved

Agents en parallèle sur worktrees

Un dépôt, **deux répertoires de travail**, deux agents **en même temps**.



i **Paralléliser demande d'isoler** : chaque agent a ses fichiers, sa branche et son contexte.

© ZENIKA 2026 | All rights reserved

Développement parallèle : le bilan

Ce qu'on gagne

TEMPS
Deux features avancent ensemble

ISOLATION
Les agents ne se perturbent pas

REVUE
Des PR plus petites, plus lisibles

Ce que ça coûte

TOKENS × N
Chaque agent a son propre contexte

CONFLITS DE MERGE
Si les features touchent les mêmes fichiers

SERVICES PARTAGÉS
Ports, base de données, dépendances

Features indépendantes seulement. Et ce qui sature d'abord, c'est **vous** : au-delà de **2 sessions**, on subit.

© ZENIKA 2026 | All rights reserved

Le coût caché : l’érosion de la connaissance



Écrire moins

Le code vous appartient moins quand vous ne l’écrivez pas.



Relire n’est pas apprendre

On valide une forme, pas un raisonnement.



Lent et invisible

On le découvre en incident, deux ans plus tard.

Ce n’est pas un argument contre l’autonomie, c’est sa **condition** : ce qu’on automatise, il faut savoir le **reprendre**.

© ZENIKA 2026 | All rights reserved

Exercice 3 — Déléguer et isoler

OPTIONNEL 15 min ↗

Le concept

Un sous-agent **isole** son contexte et ne renvoie que son résultat… mais il **coûte**.

La consigne

1. Confie **avec le mécanisme de ton harness** à des sous-agents : une exploration du code, la revue de `bookings.controller.ts`.
2. Refais-le dans la session principale, et compare.

Réussi quand…

- les deux tâches ont tourné dans des **contextes isolés** ;
- on compare **contexte consommé**, **fichiers lus** et **qualité**.

i **Débrief** : qu'a gagné la session principale ? Et combien a coûté la session entière ?

© ZENIKA 2026 | All rights reserved

Quiz : quel pattern ?

QUIZ 3 min

Spec → plan → tâches → code, chaque étape relue

chaîne

Les demandes simples vont à un modèle léger, les complexes à un modèle de raisonnement

routeur

Trois revues du même diff : sécurité, performance, lisibilité

parallélisation

Un agent écrit l'endpoint, un autre le relit avec une grille

évaluateur

TIP Superviser au lieu d'approuver : ajoute un agent seulement si tu sais qui relira son travail.

© ZENIKA 2026 | All rights reserved

Module 6 · 25 min

Au-delà du poste

Que change l’agent dans la chaîne de prod et pour l’entreprise ?



© ZENIKA 2026 | All rights reserved

De la revue locale à la revue de PR en CI

Revue locale

EXÉCUTION
Un hook sur le poste

PÉRIMÈTRE VU
Diff local et conventions

RESTITUTION
Dans la session

Revue de PR en CI

EXÉCUTION
Le runner CI, à chaque PR

PÉRIMÈTRE VU
PR complète, spec, règles d'équipe, analyse statique

RESTITUTION
Commentaire de PR et quality gate

`git diff -U10` : plus de contexte autour du changement, **beaucoup moins de faux positifs.**

© ZENIKA 2026 | All rights reserved

Pair-programming à trois

Le dev et un relecteur (deux personnes) + l'agent de revue, en même temps.

Cycle de PR asynchrone

Le dev soumet la PR

24 h d'attente

Le relecteur commente

12 h d'attente

Le dev corrige

2 personnes + 1 agent, en synchrone

L'agent de revue (CI) produit son rapport

Le dev et le relecteur trient les remarques en direct

L'agent applique les corrections

i Moins d'attente, des retours plus pertinents, une validation immédiate.

© ZENIKA 2026 | All rights reserved

Politique de merge graduée

Toutes les PR ne se valent pas : la gouvernance suit le **risque**.

DOC, CORRECTIF MINEUR

Merge automatique

Encadré par les evals et la CI.

FONCTIONNELLE

Revue synchrone

Dev + relecteur.

ARCHITECTURE, CŒUR, SÉCURITÉ

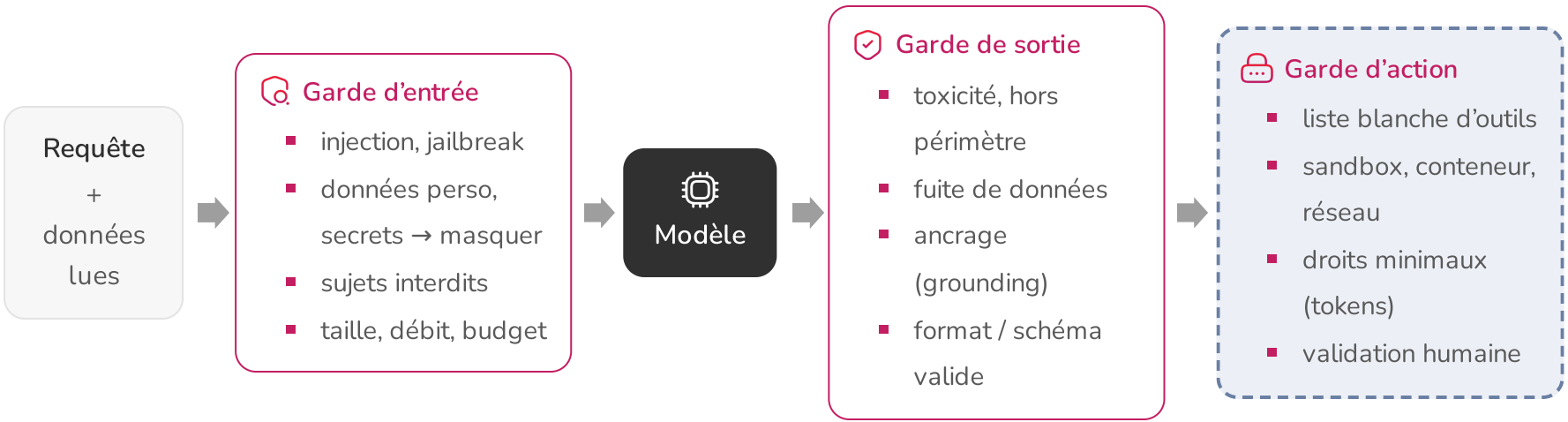
Double validation

Humaine, obligatoire.

i Les agents **préparent les preuves** ; sur le cœur de métier, **l'humain décide**.

© ZENIKA 2026 | All rights reserved

Où placer les contrôles



i Les gardes d'entrée et de sortie **réduisent** le risque. La garde d'action le **borne** : un agent sans réseau ne peut pas exfiltrer.

© ZENIKA 2026 | All rights reserved

L'AI gateway : un seul point de passage

DÉFINITION

Un proxy entre vos agents et les fournisseurs de modèles

Une seule API côté agents, N fournisseurs derrière. Aucune clé de fournisseur sur les postes.

ROUTAGE

Choix du modèle, fallback si un fournisseur tombe.

FINOPS

Quotas et plafond de dépense par équipe.

OBSERVABILITÉ

Tokens, coût et latence par appel.

EMBALLEMENT

Coupe l'agent coincé dans une boucle.

GUARDRAILS

Masquage, détection d'injection : appliqués à tous.

TRAÇABILITÉ

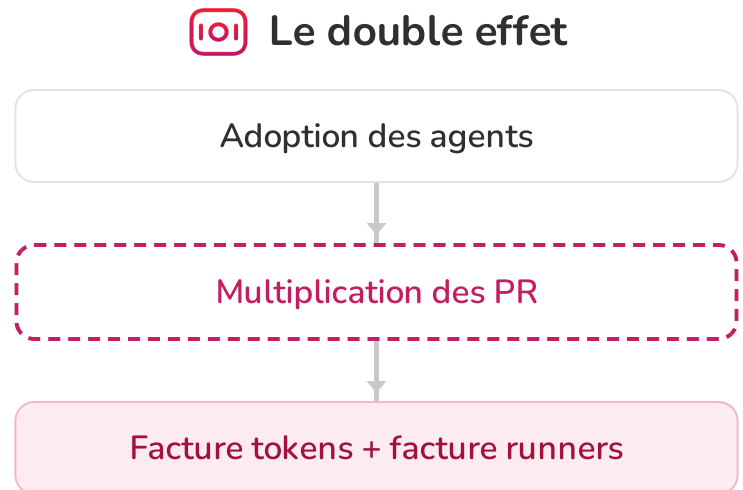
Modèles autorisés, résidence des données, audit.

© ZENIKA 2026 | All rights reserved

Le budget caché : au-delà des tokens

Plus d'agents, c'est plus de PR. Donc plus de **builds**, de **tests**, de **mutation testing**.

La facture est **double** : les tokens, et les **runners CI**.



i Un budget IA, c'est **tokens + CI + stockage + temps de revue**.

© ZENIKA 2026 | All rights reserved

Investir ou attendre : le test à trois questions

Investir

SPÉCIFIQUE ?
Contexte propre à l'équipe ou au produit
> *nos conventions de test : factories, sans mock*

RÉPÉTÉ ?
Plusieurs fois par semaine
> *lancer les tests après chaque édition*

MAINTENU ?
Un propriétaire identifié
> *la skill de revue, portée par le tech lead*

Attendre

GÉNÉRIQUE
Ce que le modèle fera bientôt nativement
> *une skill générique de tests unitaires*

RARE
Le geste qu'on fait deux fois par an
> *la migration de schéma annuelle : une note suffit*

ORPHELIN
Personne ne le maintiendra
> *la skill écrite pour un projet terminé*

Trois oui : on écrit la skill ou le hook. Un non : **on attend**.

© ZENIKA 2026 | All rights reserved

Réversibilité : éviter le vendor lock-in



Harness ≠ fournisseur

L'adhérence est dans le harness ; les modèles changent tous les trimestres.

> *choisir l'outil pour ses capacités, pas pour son modèle du moment*



Modèle via la gateway

Basculer sans réécrire l'outillage.

> *code-standard change de modèle, les postes ne bougent pas*



Formats ouverts

AGENTS.md, SKILL.md, MCP survivent à un changement d'outil.

> *les skills de l'équipe repartent telles quelles*



Résidence des données

À exiger : aucune donnée ne sert au réentraînement.

> *clause au contrat, hébergement connu*

TIP Tracé et portable : un agent qu'on peut auditer, et quitter.

© ZENIKA 2026 | All rights reserved

Clôture · 10 min

Ce qu’on garde



© ZENIKA 2026 | All rights reserved

Une requête, de bout en bout

1. **SDD**
   Une tâche issue d'une **spec relue**.
2. **Orchestrer**
   La tâche est confiée à un **agent**.
3. **Harness**
   Construit le **contexte**, préfixe stable en tête.
4. **Gateway**
   Filtre, **route** et **journalise** l'appel.
5. **Modèle**
   Prefill en cache, puis **decode**.
6. **Harness**
   Hook, permission, tool, puis il **reboucle**.

> *i* Le modèle prédit, le harness agit, la gateway gouverne.

© ZENIKA 2026 | All rights reserved

Trois idées à garder

1



Le contexte fait la facture

L'entrée domine, le préfixe stable est en cache, les sous-agents isolent sans être gratuits.

RÉFLEXE

Mesurer ce qui entre.

2



Le harness agit

Ce qui doit arriver se configure (hook, sandbox, tests), ça ne se demande pas.

RÉFLEXE

Hook plutôt que consigne.

3



La gouvernance se trace

Spec versionnée, commit tracé, appels par la gateway, revue humaine là où ça compte.

RÉFLEXE

Ce qui n'est pas tracé n'est pas gouverné.

© ZENIKA 2026 | All rights reserved

Quiz de sortie

QUIZ 3 min

1

Pourquoi brancher un MCP en cours de session coûte-t-il cher ?

il casse le cache du préfixe

2

« Lance toujours les tests » : où l'écrire pour que ce soit garanti ?

dans un hook

3

Six sous-agents, session principale à 20 % : tout va bien ?

non : regarder le total

4

Où poser un plafond de dépense pour toute l'équipe ?

dans la gateway

© ZENIKA 2026 | All rights reserved

Questions

Le modèle prédit, le harness agit, la gateway gouverne.

© ZENIKA 2026 | All rights reserved